

FlowOps

Phase 3 — Kubernetes Deployment with Helm

Phase 3 moves FlowOps from manual `go run` commands to a real Kubernetes cluster. You Dockerize all 5 services, write Helm charts, deploy to a local kind cluster, and verify the full request chain works inside K8s. This is the step that makes FlowOps look and behave like a production system.

Why Kubernetes?

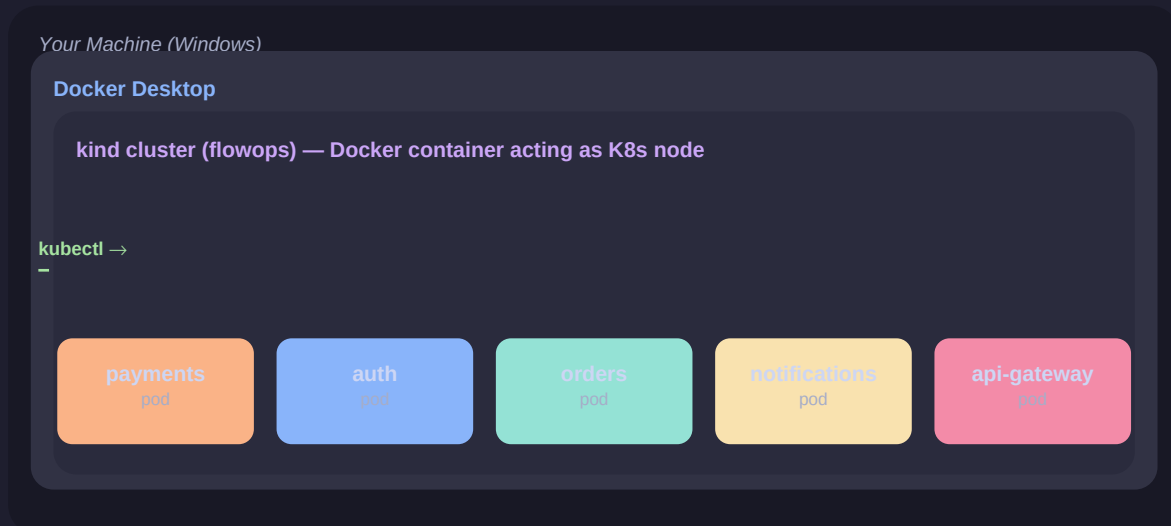
go run	Docker Compose	Kubernetes
- Manual 5 terminals - Crashes = dead - No restart - Dev only	- Semi-auto <code>docker compose up</code> - Restarts locally - No scaling - Dev/test	- Fully managed - Declarative YAML - Auto-restart - Auto-scale - Production

Kubernetes is the standard for running containerised workloads in production. It handles restarts, scaling, service discovery, and rolling updates automatically. Phase 3 gives you hands-on experience with the real thing.

1. Core concepts

kind — Kubernetes IN Docker

`kind` runs an entire Kubernetes cluster inside a Docker container on your machine. No cloud account needed. The cluster behaves exactly like a real K8s cluster. You created yours with `kind create cluster --name flowops`.



Your kind cluster is a Docker container that acts as a Kubernetes node. `kubectl` talks to it exactly like it would talk to a cloud cluster.

kubectl — the Kubernetes CLI

`kubectl` is the command line tool for interacting with Kubernetes. You use it to see pods, deployments, services, logs, and more. It reads your kubeconfig at `~/.kube/config` to know which cluster to talk to. `kind` automatically updates this file when you create a cluster.

Helm — Kubernetes package manager

Helm is like `npm` for Kubernetes. Instead of writing raw YAML files for every service, you write a Helm chart — a structured package of templates and values. `helm install` renders the templates and applies them to your cluster in one command.

Pod vs Deployment vs Service

- **Pod** — the smallest unit in K8s. One running container instance.
- **Deployment** — manages pods. Defines which image to run, how many replicas, restart policy.
- **Service** — gives pods a stable network address. Other pods reach payments at `payments:50053` regardless of which pod IP is active.

2. Dockerizing the services

Kubernetes runs containers — it can't do `go run`. Each service needs to be compiled into a binary and packaged as a Docker image. You use a **multi-stage Dockerfile** for each service.



Why multi-stage?

Stage 1 uses the full Go image (~800MB) to compile your code. Stage 2 starts fresh with just Alpine Linux (~5MB) and copies only the compiled binary. The Go compiler, source code, and build tools are thrown away. Final image is ~15MB instead of 800MB.

The Dockerfile pattern (same for all 5 services)

```
# Stage 1 — compile
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./          # copy deps first (Docker cache optimization)
RUN go mod download            # download deps (cached if go.mod unchanged)
COPY . .                      # copy entire project (shared proto code needed)
RUN go build -o payments ./services/payments

# Stage 2 — run
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/payments . # only the binary
EXPOSE 50053
CMD [\"./payments\"]
```

Only 3 things change per service: the binary name after `-o`, the build path `./services/servicename`, and the `EXPOSE` port.

Service Dockerfile reference

Service	Binary name	Build path	Port
----------------	--------------------	-------------------	-------------

payments	payments	./services/payments	50053
notifications	notifications	./services/notifications	50054
auth	auth	./services/auth	50051
orders	orders	./services/orders	50052
api-gateway	api-gateway	./services/api-gateway	8080

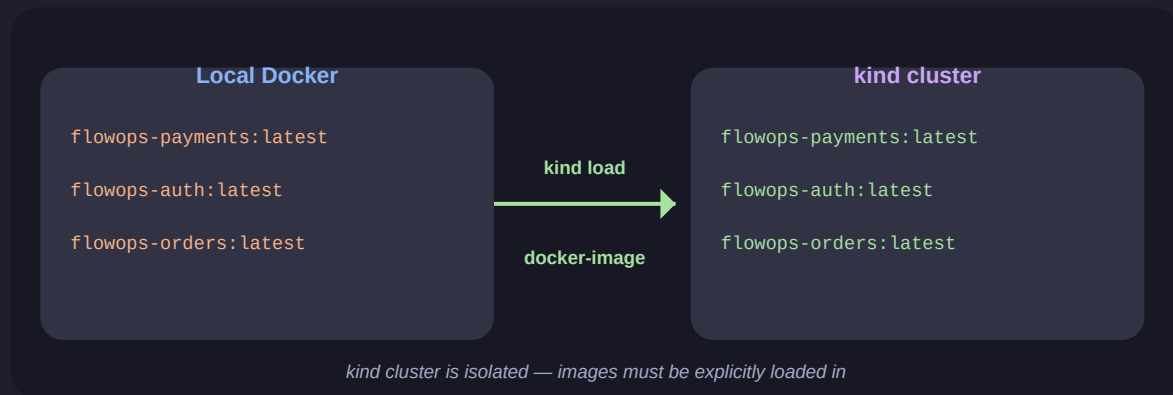
Build commands

```
# Build all 5 images from project root (. = build context)
docker build -t flowops-payments:latest      -f services/payments/Dockerfile .
docker build -t flowops-notifications:latest -f services/notifications/Dockerfile .
docker build -t flowops-auth:latest          -f services/auth/Dockerfile .
docker build -t flowops-orders:latest        -f services/orders/Dockerfile .
docker build -t flowops-api-gateway:latest   -f services/api-gateway/Dockerfile .
```

go.mod version: the Dockerfile uses `golang:1.25-alpine`. Your `go.mod` must declare `go 1.25.0` or lower. Run `go mod tidy` if the version gets bumped by VS Code.

3. Loading images into kind

Your local Docker and the kind cluster are two isolated environments. Building an image in local Docker doesn't make it available to kind. You must explicitly load each image into the cluster.



```
# Load all 5 images into the kind cluster named 'flowops'
kind load docker-image flowops-payments:latest --name flowops
kind load docker-image flowops-notifications:latest --name flowops
kind load docker-image flowops-auth:latest --name flowops
kind load docker-image flowops-orders:latest --name flowops
kind load docker-image flowops-api-gateway:latest --name flowops
```

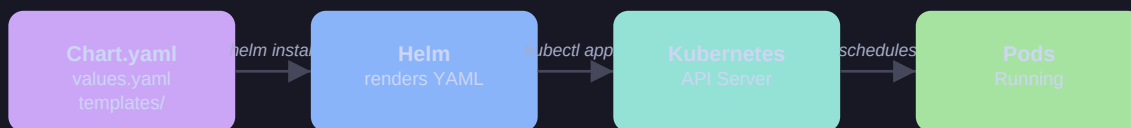
This is also why `imagePullPolicy: Never` is set in the Helm values. It tells Kubernetes: don't try to pull this image from Docker Hub — use what's already loaded locally. Without this, K8s would fail with `ImagePullBackOff`.

4. Helm charts

A Helm chart is a folder with a specific structure. `Chart.yaml` is the metadata. `values.yaml` is the config. `templates/` contains the Kubernetes YAML templates.

```
helm/flowops/  
  Chart.yaml  
  values.yaml  
  templates/  
    payments-deployment.yaml  
    payments-service.yaml  
    auth-deployment.yaml  
    auth-service.yaml  
    orders-deployment.yaml  
    orders-service.yaml  
    notifications-deployment.yaml  
    notifications-service.yaml  
    gateway-deployment.yaml  
    gateway-service.yaml
```

How helm install works



`helm install flowops ./helm/flowops → kubectl get pods → 5 pods Running`

Chart.yaml

```
apiVersion: v2  
name: flowops  
description: FlowOps GitOps engine with Go microservices  
type: application  
version: 0.1.0  
appVersion: "1.0.0"
```

values.yaml

```
image:  
  pullPolicy: Never          # use locally loaded images, not Docker Hub  
  
services:  
  payments:  
    image: flowops-payments:latest  
    port: 50053  
  auth:
```

```
    image: flowops-auth:latest
    port: 50051
orders:
  image: flowops-orders:latest
  port: 50052
notifications:
  image: flowops-notifications:latest
  port: 50054
gateway:
  image: flowops-api-gateway:latest
  port: 8080
```

5. Deployment and Service YAML

Each service needs two Kubernetes objects — a Deployment and a Service. The pattern is identical for all 5 services, only the name, image, and port change.

Deployment (payments-deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments
spec:
  replicas: 1
  selector:
    matchLabels:
      app: payments          # links Deployment to its Pods via this label
  template:
    metadata:
      labels:
        app: payments        # Pod gets this label
    spec:
      containers:
        - name: payments
          image: flowops-payments:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 50053
```

Service (payments-service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: payments            # other pods reach this service at payments:50053
spec:
  selector:
    app: payments           # routes traffic to pods with this label
  ports:
    - port: 50053           # port the Service exposes
      targetPort: 50053     # port inside the container
```

How service discovery works: inside Kubernetes, the orders service reaches payments by calling `payments:50053`. Kubernetes DNS resolves `payments` to the Service's ClusterIP automatically. This is why you updated the addresses from `localhost:50053` to `payments:50053` in the orders and gateway code.

6. Deploying and verifying

Install the Helm chart:

```
helm install flowops ./helm/flowops
```

```
# Expected output
NAME: flowops
LAST DEPLOYED: ...
STATUS: deployed
```

Verify all pods are running:

```
kubectl get pods
```

```
# Expected output
NAME                                READY   STATUS    RESTARTS   AGE
api-gateway-74c8b9867-79rt7        1/1     Running   0           18s
auth-6699886759-l7kcq              1/1     Running   0           18s
notifications-65f964bd8-5495h      1/1     Running   0           18s
orders-557cdfb9d9-n75sr            1/1     Running   0           18s
payments-76869b96c8-bpg9k         1/1     Running   0           18s
```

Forward the gateway port and test:

```
# Terminal 1 – keep this running
kubectl port-forward service/api-gateway 8080:8080
```

```
# Terminal 2 – send a request
curl.exe -X POST http://localhost:8080/order -H "Content-Type: application/json" -d "{\"token\":\"va\"}
```

```
# Expected
{"order_id":"order-1781807960","success":true}
```

Useful kubectl commands

Command	What it does
kubectl get pods	List all running pods
kubectl get deployments	List all deployments
kubectl get services	List all services and their ClusterIPs
kubectl logs <pod-name>	View logs from a pod
kubectl describe pod <pod-name>	Detailed info about a pod (debug crashes)
kubectl rollout restart deployment/x	Restart a deployment (pick up new image)
helm list	List installed Helm releases
helm uninstall flowops	Remove everything from the cluster

7. Key lessons from Phase 3

Localhost doesn't exist inside Kubernetes.

When services run in pods, they reach each other by Kubernetes service name, not localhost. `payments:50053` not `localhost:50053`. Always update addresses when moving from local to K8s.

kind needs images explicitly loaded.

kind is isolated from your local Docker. After every `docker build` you must run `kind load docker-image`. If you forget, pods will fail with `ImagePullBackOff`.

imagePullPolicy: Never is critical for local development.

Without this, Kubernetes tries to pull images from Docker Hub and fails because your images are local only. Always set this when using kind.

Deployment + Service is the standard K8s pattern.

Every workload follows the same pattern: Deployment manages the pods, Service gives them a stable address. The label `app: servicename` is the glue that links them together.

After code changes — rebuild, reload, rollout restart.

```
docker build -t flowops-payments:latest -f services/payments/Dockerfile .
kind load docker-image flowops-payments:latest --name flowops
kubectl rollout restart deployment/payments
```

Phase 3 complete. Five services running in Kubernetes, managed by Helm, with full service discovery and port forwarding.

Next: Phase 4 — GitOps reconciler engine that automatically syncs the cluster to GitHub.